

FastAPI Project · Library Management API

A minimal **FastAPI REST API** for managing a book library with **SQLite** storage and a clean **repository pattern** architecture.

· Features

- **Book CRUD** · Create, Read, Update, Partial Update, Delete books
 - **SQLite** · Lightweight persistent storage
 - **Repository Pattern** · Clean separation of concerns
 - **Pydantic Schemas** · Request/response validation
 - **Lifespan Management** · Database initialization on startup
-

.. Architecture

```
Client · FastAPI Routes · Repository · SQLite Database
      ·
      Pydantic Schemas (validation)
```

· File Structure

```
FastAPIProject/
... main.py           # FastAPI app with lifespan hook
... app/
  ... routes/books.py # GET/POST/PUT/PATCH/DELETE endpoints
  ... schemas.py      # BookBase, BookCreate, BookUpdate, Book
  ... repository.py   # Repository CRUD operations
  ... database.py     # SQLite schema initialization
... library.db        # SQLite database file
... test_main.http    # HTTP test file
... pyproject.toml
```

· Getting Started

```
cd FastAPIProject
uv sync
uv run uvicorn main:app --reload
```

Visit <http://localhost:8000/docs> for interactive Swagger API docs.

· API Endpoints

Method	Endpoint	Description
GET	/	Health check
GET	/books	List all books
GET	/books/{id}	Get a specific book
POST	/books	Create a new book
PUT	/books/{id}	Full update a book
PATCH	/books/{id}	Partial update a book
DELETE		

/books/{id} | Delete a book |

Pydantic Schemas

Schema	Purpose		
BookBase	Base fields (title, author, year)	BookCreate	Creation payload
BookUpdate	Partial update payload (all optional)	Book	Full response model with ID

· Logic Flow

- 1 **Startup** · Lifespan creates database tables
 - 2 **Request** · FastAPI validates via Pydantic schemas
 - 3 **Route** · Calls repository layer function
 - 4 **Repository** · Executes SQL against SQLite
 - 5 **Response** · Pydantic serialized JSON returned
-

· Dependencies

FastAPI, uvicorn

· License

Educational project · use freely for learning and reference.